

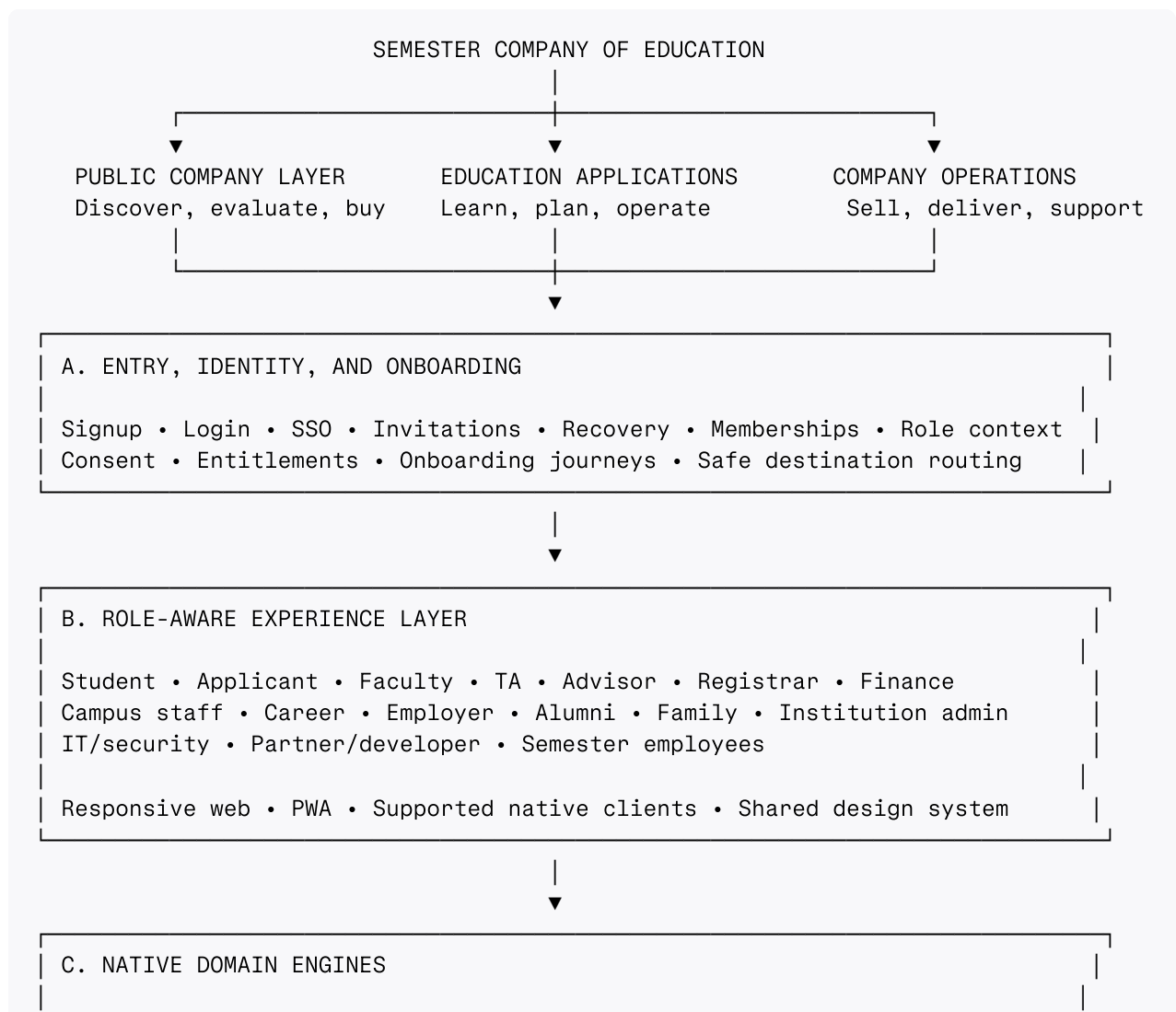


Show and break down entire semester mainframe

Semester’s **mainframe** is the complete operating architecture behind the company, its education platform, and every institutional deployment: the public website, applications, identity system, native product engines, data foundation, AI, administrative consoles, company operations, and infrastructure.

Here, “mainframe” means Semester’s central system architecture—not a claim that it runs on a traditional mainframe computer. The breakdown below describes the target architecture in your specifications; it is not verification that every component is already implemented or deployed. Your documents define Semester as one native Education Operating System with shared identity, policy, workflows, data authority, and audit controls.^{[1] [2]}

1. Entire mainframe map



Planning	Learning	Assessment	Registrar/SIS
Registration	Academic records	Advising	Student accounts
Productivity	Campus services	Community	Career/credentials
Family sharing	Commerce	Customer success	Company operations



D. SHARED PLATFORM CORE
Authorization • Policy • Workflow • Approvals • Consent • Notifications
Search • Files • Scheduling • Audit • Events • Projections • Metering
Configuration • Feature flags • API contracts • Lifecycle management



E. DATA FOUNDATION	F. AI / INTELLIGENCE	G. CONTROL PLANES
Canonical records	AI gateway	Institution controls
Education graph	Approved retrieval	Company command center
Financial ledgers	Model/provider policy	Release / incident ops
Academic records	Tool permissions	Developer governance
Files / indexes	Evaluation / provenance	Evidence / risk
Analytics projections	Human approval	Tenant configuration



H. INTEGRATION AND MIGRATION BOUNDARY
Existing SIS/LMS • Identity providers • Banks/payment rails • AI providers
Campus systems • Email/push delivery • Partner APIs • Import/export
Mapping • Sync • Reconciliation • Dual run • Cutover • Rollback



I. INFRASTRUCTURE AND TRUST FOUNDATION
Hosting • Compute • Database • Storage • Workers • Network • Secrets
CI/CD • Monitoring • Backups • Recovery • Security • Privacy
Accessibility • Tenant isolation • Retention • Incident response

This reorganizes the product domains in your native-platform specification into one system view. Your specifications separately identify the learning platform, registrar/SIS, student success, finance, campus services, AI, control plane, migration factory, trust platform, company OS, and developer marketplace.^[1]

2. Frontend and product systems

Public-to-application entry

The public company site is the front door. It explains Semester, establishes trust, captures demand, and sends each person into the correct identity and onboarding path.

```
Marketing / referral / campus invitation / employee provisioning
→ Appropriate entry page
→ Signup, SSO, invitation, demo request, or employee login
→ Identity verification
→ Membership and role resolution
→ Required consent and policy acknowledgement
→ Role-aware onboarding
→ First meaningful action
→ Daily workspace
```

The same person may hold several roles. The proposed experience should let them switch explicitly between authorized workspaces—for example, student at one institution and TA in a particular course—without merging permissions.

Education product domains

Native system	What it contains	Main workspaces
Student OS	Today, action center, tasks, calendar, schedule, goals, academic plan, registration readiness	Student, applicant
Productivity	Notes, documents, files, projects, collaboration, messaging, version history, offline-safe drafts	Students, faculty, staff
Learning	Course Studio, syllabus, objectives, resources, discussions, study tools, tutoring, office hours	Faculty, TA, student
Assessment	Assignments, question banks, attempts, rubrics, accommodations, grading, moderation, release, regrades	Faculty, TA, student
Registrar/SIS	Institution hierarchy, terms, catalog, sections, enrollment, prerequisites, holds, waitlists, overrides	Registrar, academic affairs
Academic records	Grades, standing, transfer credit, degree audit, graduation, credentials, transcripts	Registrar, records staff
Student success	Caseloads, appointments, check-ins, plans, referrals, outreach, support, aggregate outcomes	Advisor, support, leadership
Student finance	Charges, credits, statements, balances, payment plans, refunds, reconciliation	Student, accounts staff
Campus services	Housing, dining, library, spaces, events, transit, jobs, service directory, service cases	Students, service operators
Community	Organizations, groups, posts, study sessions, reports, moderation, appeals	Members, moderators
Career and lifelong learning	Skills evidence, portfolio, opportunities, applications, employers, mentoring, alumni, credentials	Student, employer, alumni

Native system	What it contains	Main workspaces
Family sharing	Invitations, scoped grants, expiry, revocation, access history	Student, guardian

These domains are explicitly included in the native-platform plan. Their inclusion in the plan establishes intended scope, not production readiness. ^[1]

Three distinct console families

Console family	Operated by	Purpose	Access boundary
Institution consoles	Registrar, advisors, finance, campus staff, IT, institutional leaders	Operate the institution's education and services	Authorized institution and resource scope
Semester company consoles	Sales, implementation, support, finance, product, engineering, people operations	Operate Semester as a company and deliver deployments	Employee capabilities and approved customer access
Developer/partner consoles	Developers, partner administrators, certification staff	Build, install, govern, and support integrations/apps	Sandbox, application, organization, and approved scopes

Your role specification distinguishes institutional administration from Semester's Operations Command Center. Semester employment must not automatically grant access to all institution records. ^[2]

3. The shared platform core

The mainframe's most important part is not its dashboards. It is the shared machinery that makes every dashboard behave consistently.

Core engines

Engine	Responsibility	Example
Identity	Establish account identity and authentication lifecycle	Verify sign-in or accept institution SSO
Membership	Connect accounts to institutions and organizations	Confirm active faculty membership
Authorization	Evaluate capabilities, scope, relationships, and conditions	Permit grading only for assigned courses
Policy	Apply institution, course, platform, and workflow rules	Enforce registration window
Consent	Track applicable grants, versions, expiry, and revocation	Limit guardian visibility
Workflow	Move work through defined states and accountable owners	Route a prerequisite override request
Approval	Require authorized review before execution	Approve an academic-record correction
Command	Execute validated changes with duplicate-request protection	Apply an approved credit once
Event	Publish facts about completed changes	<code>enrollment.confirmed</code>

Engine	Responsibility	Example
Projection	Build role-appropriate views from authoritative records	Update student schedule and registrar queue
Audit	Record who did what, when, why, and under which authority	Preserve grade-change history
Notification	Deliver relevant messages with preferences and policy	Send enrollment confirmation
Search/files	Retrieve permitted content and manage document access	Search approved course resources
Entitlements/metering	Control purchased modules and measured usage	Enable contracted features and AI quota
Lifecycle	Handle role changes, retention, export, and offboarding	End a TA's course access

Your specifications require shared workflow, policy, capability, audit, event-outbox, projection, source-status, and lifecycle mechanisms instead of duplicated implementations per domain. ^[2]
^[1]

One controlled transaction path

A proposed sensitive operation should follow this sequence:

```
User requests action
→ Authenticate
→ Resolve tenant and active membership
→ Check capability and resource relationship
→ Evaluate policy, consent, entitlement, and workflow state
→ Read authoritative records
→ Show impact preview when appropriate
→ Obtain required approval or stronger authentication
→ Execute validated command
→ Commit record change, audit record, and event together
→ Update role-specific views
→ Notify permitted recipients
→ Return receipt and remediation options
```

Not every action requires human approval. The approval engine should distinguish ordinary permitted actions from actions that require independent review.

Registration example

```
Student requests a section
→ Registration engine checks window, holds, requirements, and capacity
→ Result: enroll / waitlist / deny / request override
→ Authoritative enrollment change is committed
→ Audit and enrollment event are recorded
→ Student schedule and academic plan refresh
→ Faculty roster refreshes
```

```
→ Registrar sees the updated operational state
→ Student receives a receipt
```

This flow is specified in your registrar architecture. If an external SIS remains authoritative, Semester must show that boundary and use the approved handoff or integration rather than imply it finalized enrollment itself. ^[1]

Data foundation

The proposed canonical graph groups records by domain:

```
Identity:
Person • Account • Membership • Role • Capability

Institution:
Institution • Campus • School • Department • Program • Term

Academic:
Course • Section • Enrollment • Requirement • Grade • Academic record

Learner:
Plan • Task • Assignment • Learning evidence • Skill • Credential

Services:
Case • Appointment • Referral • Service • Booking • Community

Commercial:
Customer • Contract • Product • Price • Subscription • Invoice • Payment

Governance:
Policy • Consent • Approval • Audit event • Risk • Incident

Platform:
Integration • Migration • Release • Feature configuration • AI interaction
```

The Education Graph specification also requires authority, source, freshness, classification, version, audit history, retention, export, and deletion metadata. ^[1]

Two boundaries must remain explicit:

- A shared graph does not mean every user can query every record.
- A canonical account does not mean all institution-held records become a single unrestricted personal profile.

AI and financial boundaries

The AI platform should sit beside the domain engines, not bypass them.

```
AI request
→ User and tenant authorization
→ Data classification and applicable policy
```

- Approved source retrieval
- Permitted model/provider
- Answer or proposed action
- Source and uncertainty labels
- Authorized domain command if action is permitted
- Audit and evaluation

Your specification forbids AI from independently issuing official grades, changing academic records, enrolling/dropping students, approving financial actions, or overriding institutional policy.^[1]

For finance, the proposed native system should separate:

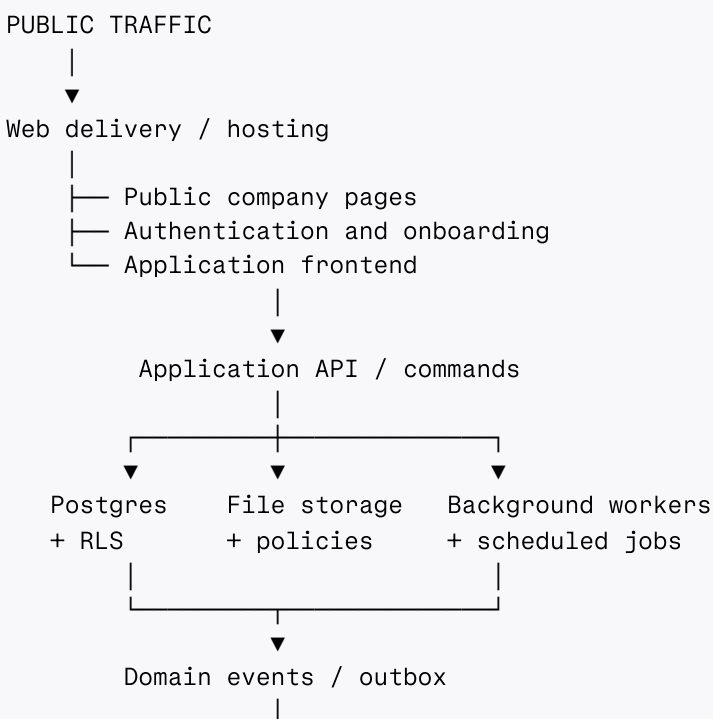
Semester's company billing
Institution student-account operations
Payment execution and settlement
Statutory accounting/reporting

Native checkout, invoicing, account ledgers, subscriptions, entitlements, and reconciliation should not collapse into one ambiguous balance. Your original plan includes payment-provider handoffs and says not to hold raw card data.^[1]

4. Infrastructure and control planes

Technical deployment view

Your documented stack proposes React/TypeScript, Supabase/Postgres, server workers or Edge Functions, and Vercel environments. Those are documented choices—not a live infrastructure audit.^[1]





The architecture should define component boundaries and contracts first. One unified platform does not require one enormous process or database table, nor does it require turning every feature into a separately deployed service.

Institution control plane

This configures how a specific institution uses Semester:

- Institution hierarchy and organizational scope.
- SSO, provisioning, memberships, delegated administration, and access reviews.
- Enabled modules, plans, entitlements, feature cohorts, and kill switches.
- Academic, AI, consent, retention, and governance policies.
- Integration connections, mapping, freshness, reconciliation, and migration status.
- Audit, assurance evidence, incident notices, portability, and offboarding. ^[1]

Semester operations control plane

This runs the company and platform:

- Leads, opportunities, quotes, contracts, pilots, and renewals.
- Tenant/customer views, implementation projects, training, support, and customer success.
- Product roadmap, releases, flags, rollouts, service health, and incidents.
- Security, privacy, accessibility, AI governance, vendor risk, and evidence.
- Billing, collections, forecasts, budgets, people operations, and board reporting. ^[2] ^[1]

A founder's executive dashboard should provide governed oversight—not an automatic unrestricted route into student, HR, financial, or security records.

Integration and migration boundary

Not every external dependency has the same purpose:

Dependency type	Proposed treatment
Legacy SIS/LMS	Controlled transition bridge where Semester intends to replace the domain
Institutional identity provider	Supported identity relationship; not necessarily something to replace
Bank/payment network	Financial infrastructure boundary, not simply legacy software
AI model provider	Replaceable compute/provider dependency behind Semester governance
Email/push delivery	Delivery infrastructure behind native messaging logic
Partner application	Deliberate ecosystem extension with approved scopes

The native-platform documents require mapping, versioning, sync monitoring, reconciliation, dual run, cutover, rollback, export, and legacy retirement for replacement domains. ^[1]

Trust surrounds every layer

Security, privacy, accessibility, tenant isolation, observability, backups, recovery, and retention are cross-cutting requirements—not a final module added after construction.

The trust plan includes asset/data inventories, access reviews, vulnerability management, incident command, control evidence, vendor risk, restore testing, business continuity, data-rights workflows, and public trust materials. ^[1]

5. Operating and build sequence

Whole-company operating loop

Marketing generates qualified demand

→ Sales scopes a truthful offer

→ Contract defines modules and responsibilities

→ Implementation provisions the tenant

→ Identity and migration establish authorized data access

→ Training and onboarding activate users

→ Product engines deliver education workflows

→ Support and reliability teams maintain continuity

→ Governance records evidence and resolves risk

→ Customer success measures agreed outcomes

→ Renewal, expansion, or governed offboarding

The lifecycle continues for every person:

Student → term progression → graduation/transfer → alumni or export

Faculty/TA → teaching assignment → term close → scoped access sunset

Advisor → caseload assignment → reassignment → access change

Guardian → consent grant → expiry/revocation

Employee → provisioning → access review → role change → offboarding

Developer → sandbox → certification → installation → retirement

These lifecycle distinctions are included in your role-system specification. ^[2]

Dependency-ordered construction

Stage	Build	Evidence required to advance
1. Reconcile	Existing code, schemas, routes, roles, deployments, specifications	Canonical model and gap register
2. Establish foundation	Identity, memberships, capabilities, policy, audit, shared shell	Authorization and tenant-isolation tests
3. Complete first workflow	Student registration-readiness journey with advisor/registrar handoff	End-to-end acceptance and first-value evidence

Stage	Build	Evidence required to advance
4. Operate deployment	Institution console, implementation, support, release controls	Support, incident, rollout, and recovery readiness
5. Expand domains	Course Studio, student success, finance, campus, career, community	Domain-specific tests and operating owners
6. Transfer authority	Approved academic/financial replacement domains	Reconciliation, cutover approval, rollback/remediation
7. Extend ecosystem	Developer platform, marketplace, regional capabilities	Scope review, partner governance, operational capacity

This ordering is an implementation proposal derived from your native-first requirements. The specifications explicitly prohibit calling a domain native-authoritative until functional parity, data authority, security/privacy, accessibility, isolation, migration integrity, institutional approval, support readiness, portability, and outcome proof are established.^[1]

Mainframe readiness register

Every major component should have one accountable record:

```

Component / domain
Operating owner
Canonical code and data
User roles and tenant scope
Authority and external dependencies
Policies and approval rules
Audit and monitoring
Test evidence
Support procedure
Recovery / migration plan
Current readiness status
Next blocking action

```

Use separate labels for design maturity, implementation maturity, deployment readiness, and record authority. A polished screen can be implemented but not pilot-ready; a pilot-ready service can still depend on an external authoritative record system.

Semester's mainframe is therefore one connected company-and-education architecture: multiple role workspaces, native domain engines, shared governed services, explicit data authority, institution and company control planes, and a reliable infrastructure foundation.

✱✱

1. now-break-down-entire-semester-infrastructure-and-sys.pdf
2. depict-semester-flow-for-every-role-whether-you-work.pdf